Transition Systems and Model Checking

Kais Klai

Professeur des Universités, LIPN
Université Sorbonne Paris Nord

kais.klai@lipn.univ-paris13.fr

Outline

1. Context
2. Model Checking
3. Formalisms and Notations
4. Formal Specifications
   - Petri nets
   - Linear Temporal Logic (LTL)
5. LTL Model Checking
   - Büchi Automata
   - Automata-Theoretic Explicit LTL Model Checking
6. Model Checking CTL
   - Binary Diagram Decision
   - Verification Algorithm for CTL
7. Conclusion

Outline

1. Context
2. Model Checking
3. Formalisms and Notations
4. Formal Specifications
   - Petri nets
   - Linear Temporal Logic (LTL)
5. LTL Model Checking
   - Büchi Automata
   - Automata-Theoretic Explicit LTL Model Checking
6. Model Checking CTL
   - Binary Diagram Decision
   - Verification Algorithm for CTL
7. Conclusion

Context



Some Properties

- Reachability: A certain situation can be reached
  x may be zero, each instruction can be executed
- Invariant: Each state respects some good property
  x is never equal to zero, an array never overflows
- Safety: Something bad can never happen
  I access the file if I enter the correct PIN
- Liveness: Something good can always happen
  the program terminate, the message will eventually arrive to the destination, the program always returns to the initial state
- Fairness: Something good happens infinitely often
  If a process asks to enter to a critical section infinitely often, it will access it infinitely often
- ...

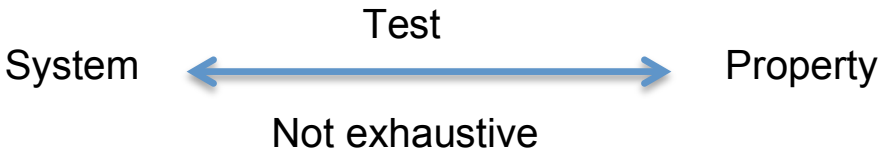
Context



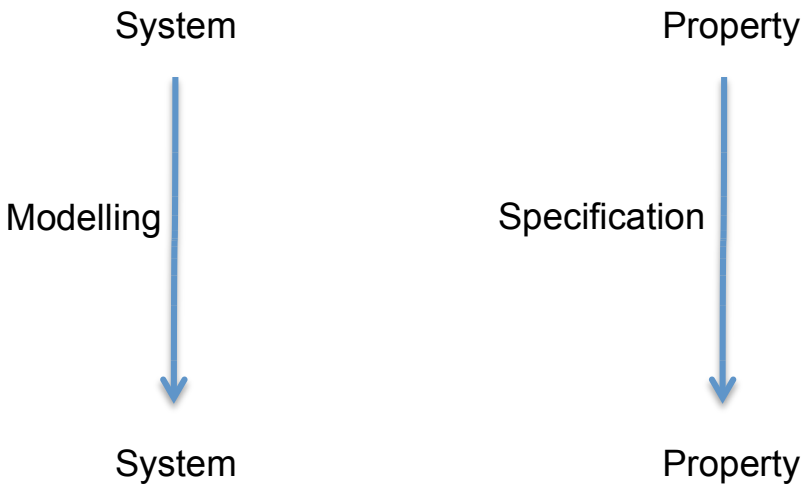
Context



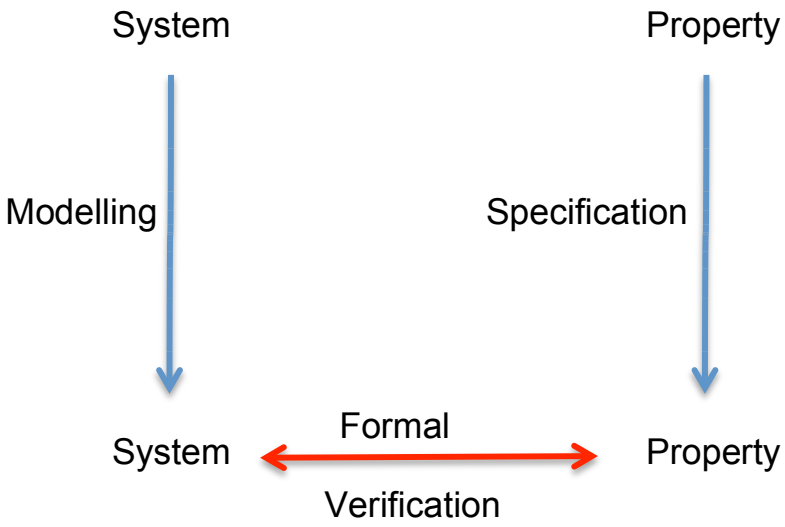
Context



Context


System
Modelling
System
Property
Specification
Property

Context


System
Property
Modelling
Specification
System
Formal
Verification
Property

Formal Verification

1. Theorem Proving
   - Logical description of the system
   - Prove properties by deduction
   - Not fully automatic

Formal Verification

1. Theorem Proving
   - Logical description of the system
   - Prove properties by deduction
   - Not fully automatic
2. Model Checking
   - Exhaustive verification
   - Fully automatic
   - Counter-examples

Example: Mutual Exclusion Algorithm

Global variables: req_P and req_Q

Process P

1. $req_P \leftarrow 1$
2. `wait(`$req_Q = 0$`)`
3. `Critical Section`
4. $req_P \leftarrow 0$

Process Q

1. $req_Q \leftarrow 1$
2. `wait(`$req_P = 0$`)`
3. `Critical Section`
4. $req_Q \leftarrow 0$

Initial state: $req_P = req_Q = 0$

Example: Mutual Exclusion Algorithm

Global variables: req_P and req_Q

Process P

1. $req_P \leftarrow 1$
2. `wait(`$req_Q = 0$`)`
3. `Critical Section`
4. $req_P \leftarrow 0$

Process Q

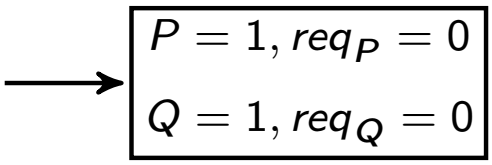
1. $req_Q \leftarrow 1$
2. `wait(`$req_P = 0$`)`
3. `Critical Section`
4. $req_Q \leftarrow 0$

Initial state: $req_P = req_Q = 0$

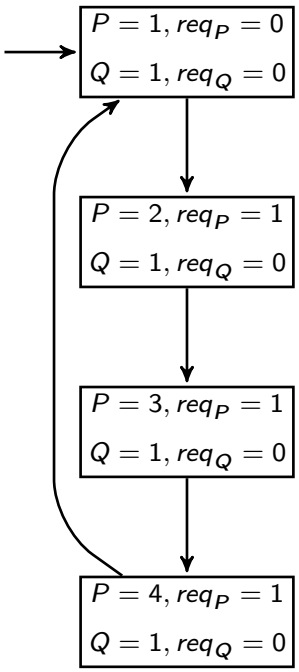
Properties to be checked:

1. Mutual exclusion
2. Fairness
3. Order

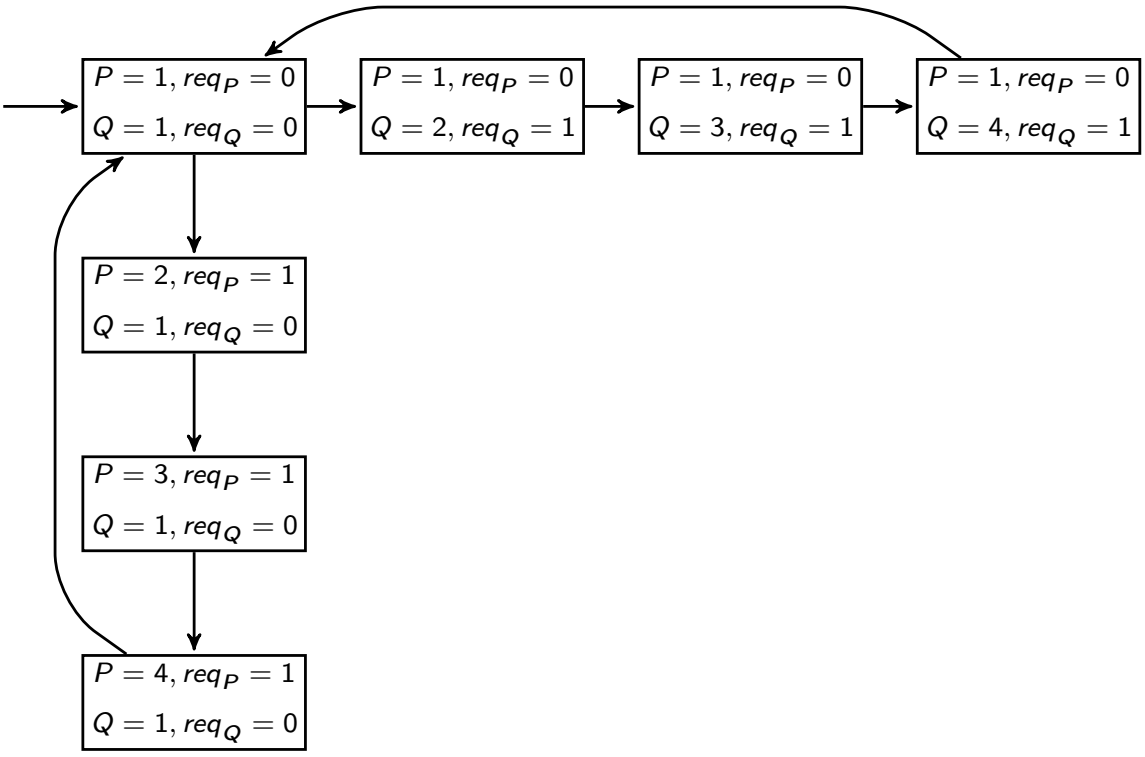
Example: Reachability State Space



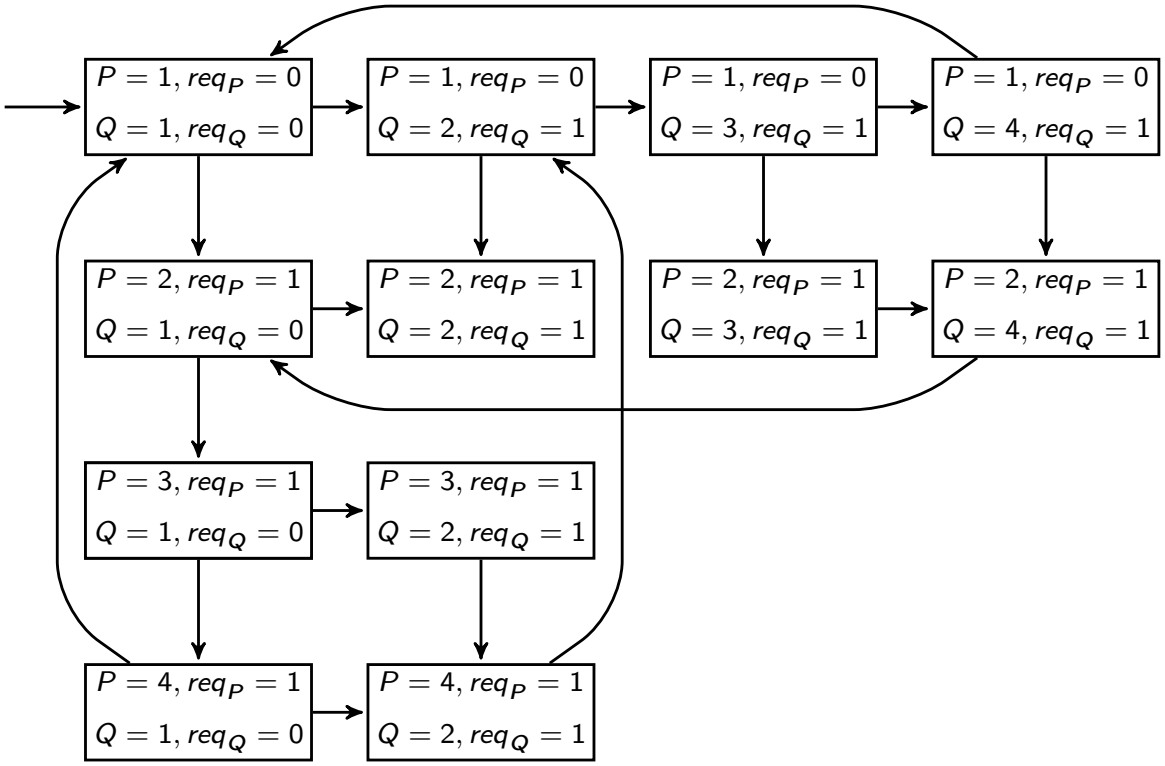
Example: Reachability State Space



Example: Reachability State Space



Example: Reachability State Space



Property 1: Mutual Exclusion

Property 1: Mutual Exclusion

We never have $P = 3 \wedge Q = 3$

Property 1: Mutual Exclusion

We never have $P = 3 \wedge Q = 3$

That's true

Property 1: Mutual Exclusion

We never have $P = 3 \wedge Q = 3$

That's true

To check this property we browse the set of reachable states. We need reachable states only, not the transitions between states.

Property 2: Fairness

Property 2: Fairness

Each path starting at a state where $P = 2$ traverses a state where $P = 3$, and the same for Q

Property 2: Fairness

Each path starting at a state where $P = 2$ traverses a state where $P = 3$, and the same for Q

That's false: State $(P = 2,\ req_P = 1,\ Q = 2,\ , req_Q = 1)$ has no successor

Property 2: Fairness

Each path starting at a state where $P = 2$ traverses a state where $P = 3$, and the same for Q

That's false: State $(P = 2,\ req_P = 1,\ Q = 2,\ , req_Q = 1)$ has no successor

To check this property we browse the reachability graph (having the reachable states only is not sufficient).

Property 3: Order

Property 3: Order

Each path starting at a state where $P = 2 \wedge Q = 1$ do not visit a state satisfying $Q = 3$ before visiting a state where $P = 3$
(+ a symmetric property for Q).

Property 3: Order

Each path starting at a state where $P = 2 \wedge Q = 1$ do not visit a state satisfying $Q = 3$ before visiting a state where $P = 3$
(+ a symmetric property for Q).

That's false: Starting from $(P = 2,\ req_P = 1,\ Q = 1,\ req_Q = 0)$, there exists a path where $P = 3$ is never satisfied.

Property 3: Order

Each path starting at a state where $P = 2 \wedge Q = 1$ do not visit a state satisfying $Q = 3$ before visiting a state where $P = 3$
(+ a symmetric property for Q).

That's false: Starting from $(P = 2,\ req_P = 1,\ Q = 1,\ req_Q = 0)$, there exists a path where $P = 3$ is never satisfied.

To check this property we browse the reachability graph (having the reachable states only is not sufficient).

Outline

1. Context
2. Model Checking
3. Formalisms and Notations
4. Formal Specifications
   - Petri nets
   - Linear Temporal Logic (LTL)
5. LTL Model Checking
   - Büchi Automata
   - Automata-Theoretic Explicit LTL Model Checking
6. Model Checking CTL
   - Binary Diagram Decision
   - Verification Algorithm for CTL
7. Conclusion

Model checking of finite state systems

Principle

1. Design the system with a model $\mathcal{M}$ and design a property φ
2. $\mathcal{M} \models \varphi$? if no, a counter-example σ
3. Analyse the result:
   - If yes, **OK**
   - If no, refine $\mathcal{M}$ using σ and go to (1).

Model checking of finite state systems

Principle

1. Design the system with a model $\mathcal{M}$ and design a property φ
2. $\mathcal{M} \models \varphi$? if no, a counter-example σ
3. Analyse the result:
   - If yes, **OK**
   - If no, refine $\mathcal{M}$ using σ and go to (1).

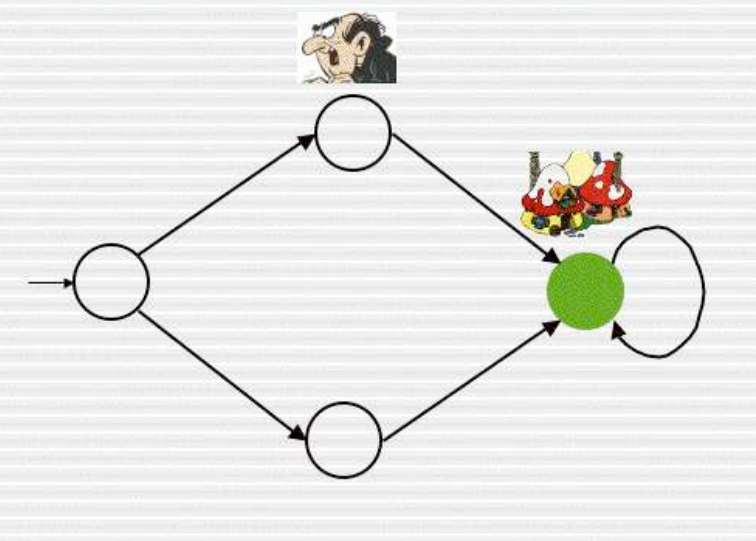
Approach

- State space traversal (Labeled Transition System)

Example

Gargamel !!!

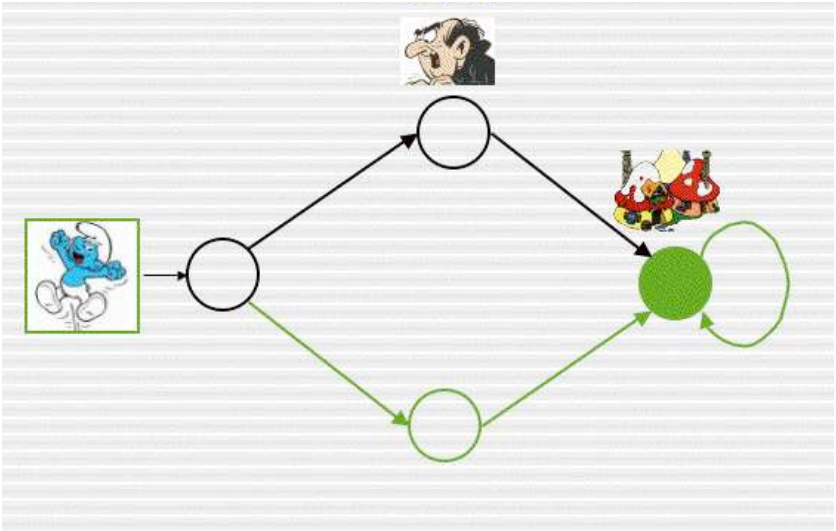
Is there any safe path?



Example

Gargamel !!!

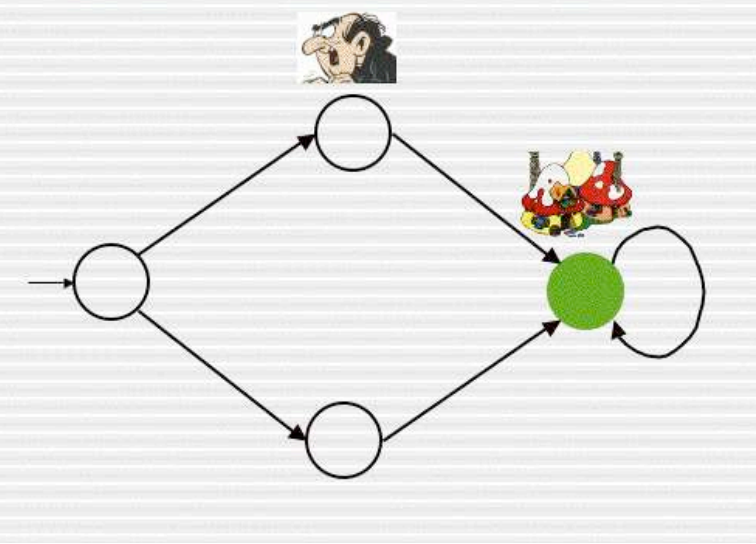
YES



Example

Gargamel !!!

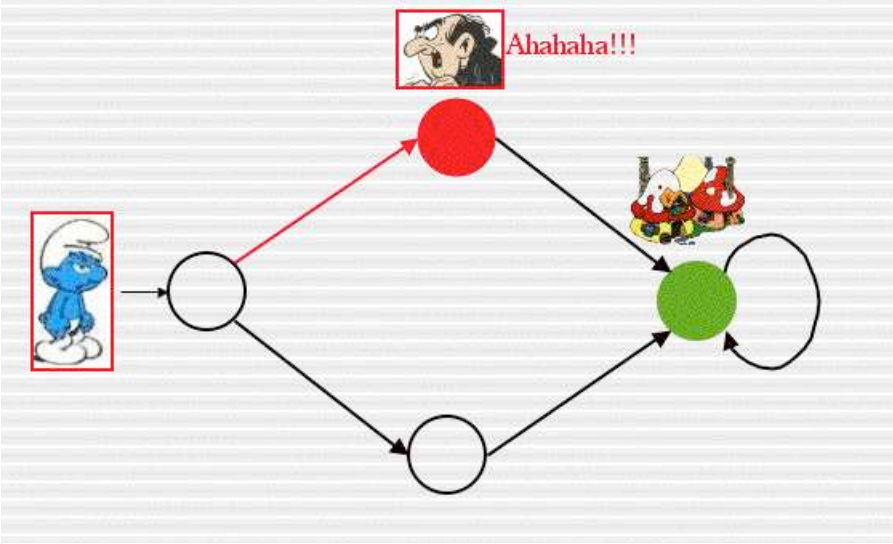
Are all the paths safe?



Example

Gargamel !!!

NO



Formal Specifications

1. The System
   Systems are formally expressed using:
   - State Machines
   - Automata
   - Petri Nets

Formal Specifications

1. The System
   Systems are formally expressed using:
   - State Machines
   - Automata
   - Petri Nets
2. The properties
   Properties are formally expressed using temporal logics
   - Linear Temporal Logic (LTL)
   - Tree Computational Logic (CTL)
   - CTL*

Formal Specifications

1. The System
   Systems are formally expressed using:
   - State Machines
   - Automata
   - Petri Nets
2. The properties
   Properties are formally expressed using temporal logics
   - Linear Temporal Logic (LTL)
   - Tree Computational Logic (CTL)
   - CTL*

Advantages:

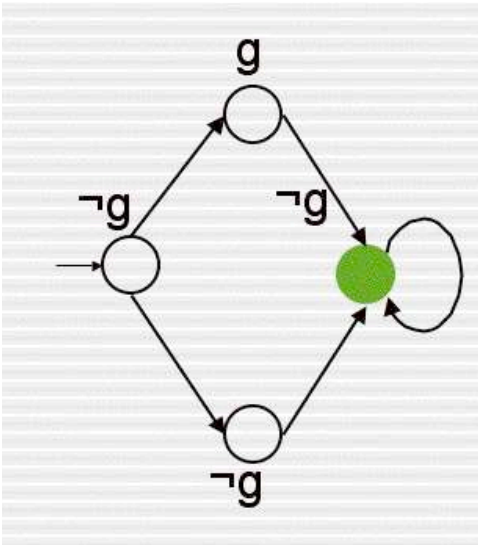
- unambiguous
- generic
- allows for automatic verification

Example

Let's be serious 5 minutes

Example

- A: for all paths
- E: there exists a path
- G: always
- g: *Gargamel*
- $\neg$: negation



The formula $EG\ \neg g$ is satisfied by the model

Model Checking

Ingredients

- $\mathcal{M} =$ The behavior of the System
- $\varphi =$ a temporal formula
- $MC = \mathcal{M} \models \varphi$?

Model Checking

Ingredients

- $\mathcal{M} =$ The behavior of the System
- $\varphi =$ a temporal formula
- $MC = \mathcal{M} \models \varphi$?

Advantages

- During specification/design time
- Automatic
- Global w.r.t. Test
- Efficient (in some fields)

Model Checking

Ingredients

- $\mathcal{M} =$ The behavior of the System
- $\varphi =$ a temporal formula
- $MC = \mathcal{M} \models \varphi$?

Advantages

- During specification/design time
- Automatic
- Global w.r.t. Test
- Efficient (in some fields)

Drawbacks

- Finite LTSs
- Requires formal expertise
- State space explosion problem

State space explosion problem

Reduction Techniques

- On-the-fly construction
  - **Stop the exploration as soon as a counter-example is found**

State space explosion problem

Reduction Techniques

- On-the-fly construction
  - Stop the exploration as soon as a counter-example is found
- Partial order reduction
  - Exploits the independence between actions

State space explosion problem

Reduction Techniques

- On-the-fly construction
  - Stop the exploration as soon as a counter-example is found
- Partial order reduction
  - Exploits the independence between actions
- Stuttering equivalence
  - stutter-invariant formula
    - $a\bar{b}.a\bar{b}.a\bar{b}.ab.ab.ab\ldots$
    - $a\bar{b}.a\bar{b}.\bar{ab}.a\bar{b}.ab.\bar{ab}.ab.\ldots$

State space explosion problem

Reduction Techniques

- On-the-fly construction
  - Stop the exploration as soon as a counter-example is found
- Partial order reduction
  - Exploits the independence between actions
- Stuttering equivalence
  - stutter-invariant formula
    - $a\bar{b}.a\bar{b}.a\bar{b}.ab.ab.ab\ldots$
    - $a\bar{b}.a\bar{b}.\bar{ab}.a\bar{b}.ab.ab.ab\ldots$
- Modularity

State space explosion problem

Reduction Techniques

- On-the-fly construction
  - Stop the exploration as soon as a counter-example is found
- Partial order reduction
  - Exploits the independence between actions
- Stuttering equivalence
  - stutter-invariant formula
    - $a\bar{b}.a\bar{b}.a\bar{b}.ab.ab.ab\ldots$
    - $a\bar{b}.a\bar{b}.\bar{ab}.a\bar{b}.ab.ab.ab\ldots$
- Modularity
- Symbolic representations (e.g., BDDs)
- ...

State space explosion problem

Reduction Techniques

- On-the-fly construction ✓
  - Stop the exploration as soon as a counter-example is found
- Partial order reduction
  - Exploits the independence between actions
- Stuttering equivalence ✓
  - stutter-invariant formula
    - $a\bar{b}.a\bar{b}.a\bar{b}.ab.ab.ab\ldots$
    - $a\bar{b}.a\bar{b}.\bar{ab}.a\bar{b}.ab.\bar{ab}.ab.\ldots$
- Modularity ✓
- Symbolic representations (e.g., BDDs) ✓
- ...

Outline

1. Context
2. Model Checking
3. Formalisms and Notations
4. Formal Specifications
   - Petri nets
   - Linear Temporal Logic (LTL)
5. LTL Model Checking
   - Büchi Automata
   - Automata-Theoretic Explicit LTL Model Checking
6. Model Checking CTL
   - Binary Diagram Decision
   - Verification Algorithm for CTL
7. Conclusion

State Machines

Syntactical Representation of a System

S=(C,V,A,T)

- C: Control States
- V: Variables
- A: Actions on V
- T: Transitions

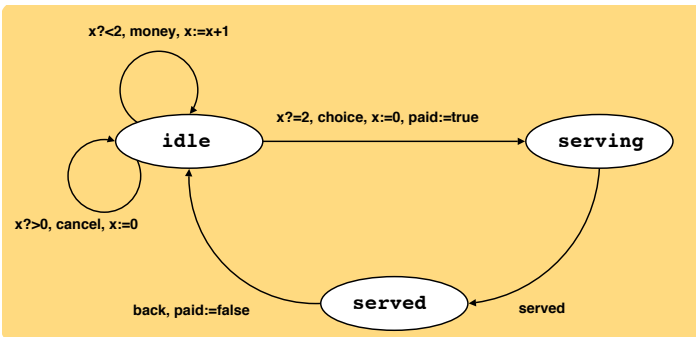
State Machines

Syntactical Representation of a System

S=(C,V,A,T)

- C: Control States
- V: Variables
- A: Actions on V
- T: Transitions





Labeled Transition System (LTS)

LTS = Semantics of the system

S=(Q,T,$\rightarrow$)

- Q: set of states (control state,valeriable's values)
- T: set of transitions
- $\rightarrow \subseteq Q \times T \times Q$: the transition relation
- we can add an initial state *I*

Labeled Transition System (LTS)

LTS = Semantics of the system

S=(Q,T,$\rightarrow$)

- Q: set of states (control state,valeriable's values)
- T: set of transitions
- $\rightarrow \subseteq Q \times T \times Q$: the transition relation
- we can add an initial state I

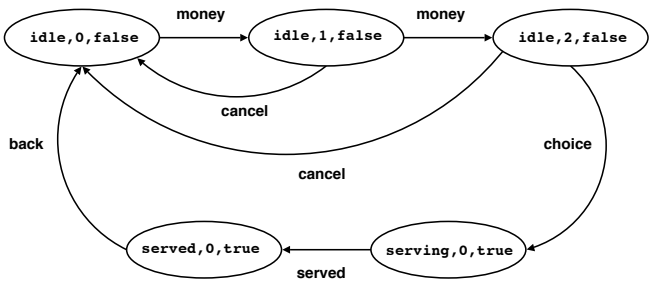
Q represents the possible states of the system
a transition t can be executed at state a leading to state q' is $(q, t, q') \in \rightarrow$ (denoted by $q \xrightarrow{t} q'$)

Labeled Transition System (LTS)

LTS = Semantics of the system

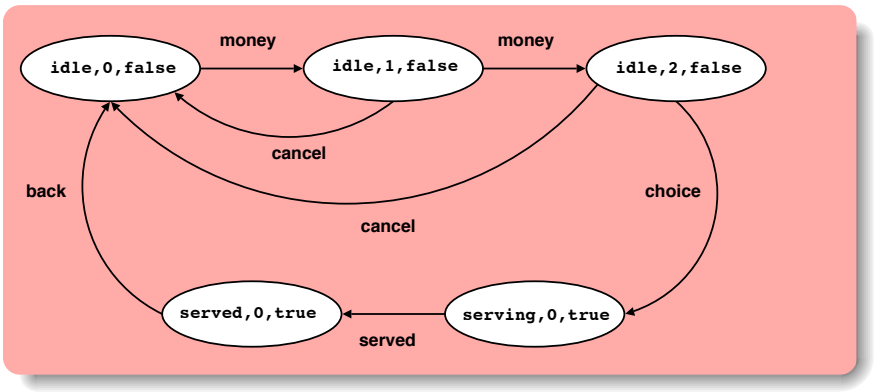
S=(Q,T,$\rightarrow$)

- Q: set of states (control state,valeriable's values)
- T: set of transitions
- $\rightarrow \subseteq Q \times T \times Q$: the transition relation
- we can add an initial state I



Q represents the possible states of the system
a transition t can be executed at state a leading to state q' is $(q, t, q') \in \rightarrow$ (denoted by $q \xrightarrow{t} q'$)

Executions of the system

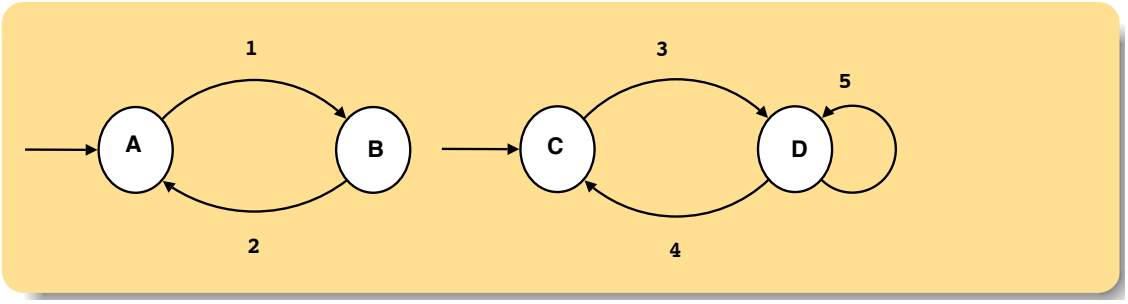


- $(i, 0, f) \xrightarrow{money} (i, 1, f) \xrightarrow{money} (i, 2, f) \xrightarrow{choice} (sg, 0, t) \ldots$
- money, money, choice, served, back, money

$L(S)$ = Language of S= The set of executions of S

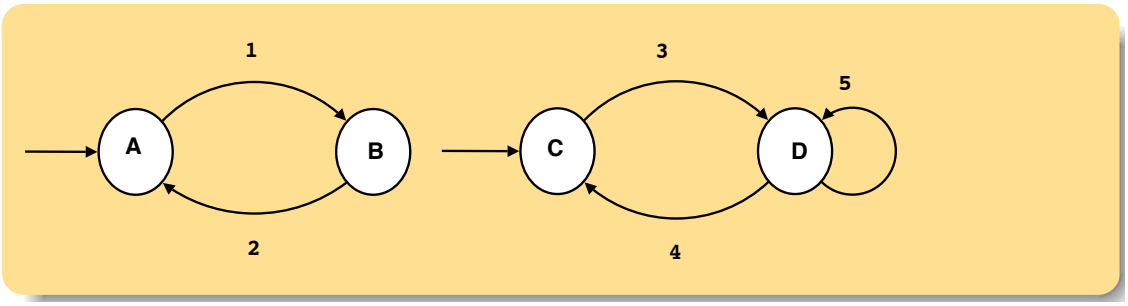
Concurrency

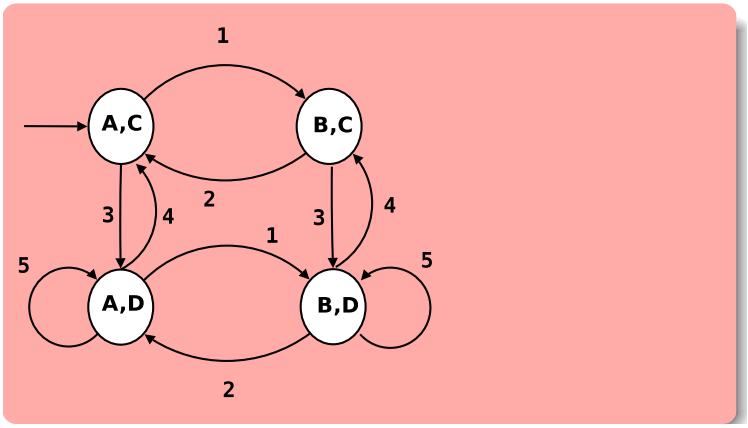
Asynchronous product



Concurrency

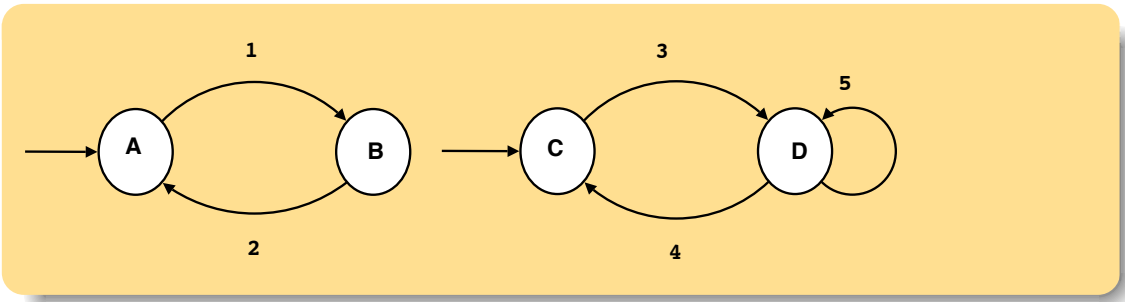
Asynchronous product





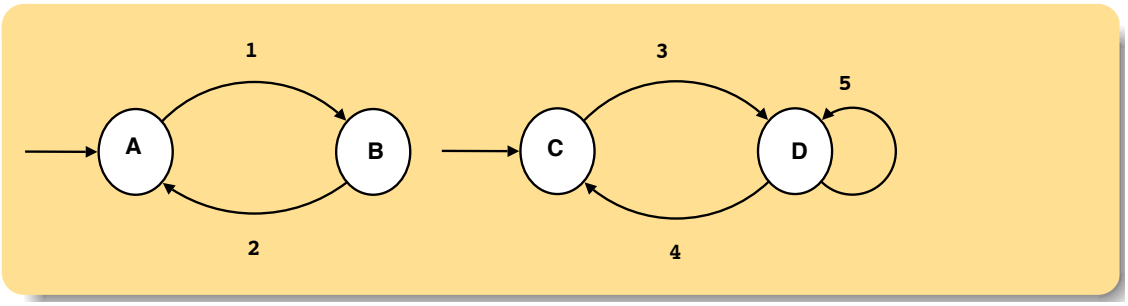
Concurrency

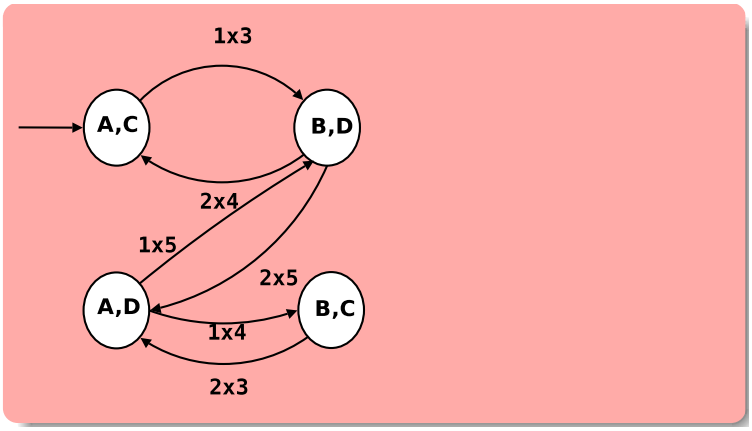
Synchronous product



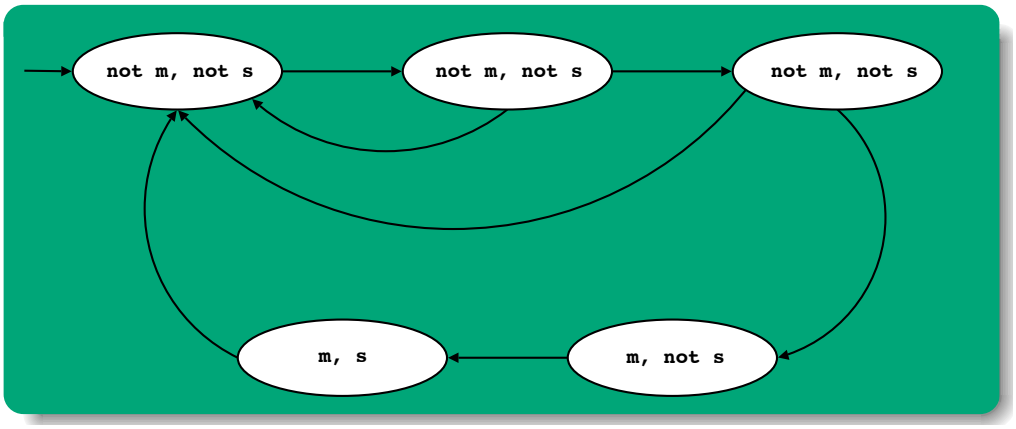
Concurrency

Synchronous product

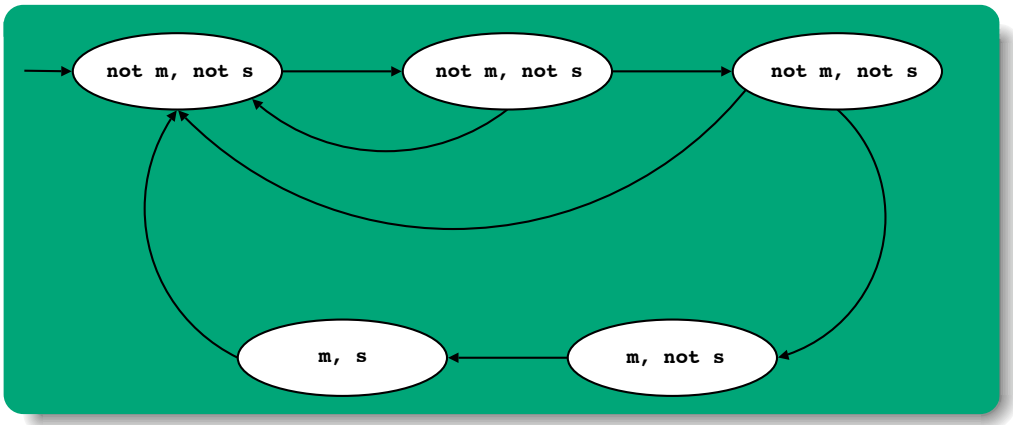


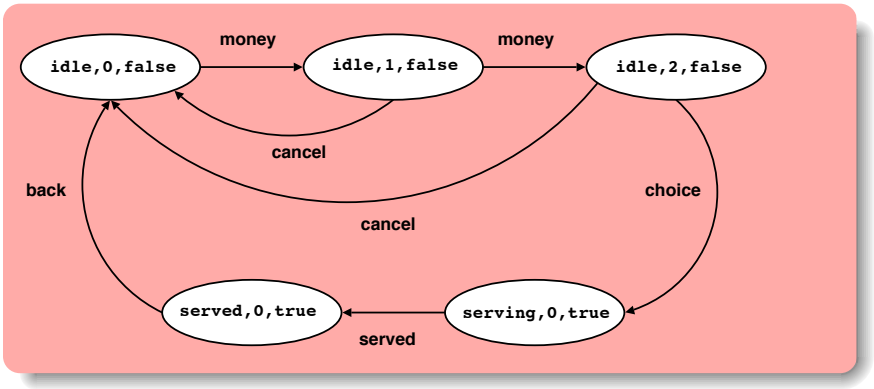


Kripke structure



Kripke structure





Exercice: The Lift Example

Le système de contrôle d'un ascenceur (pour 3 étages) est défini par :

1. le contrôleur garde en mémoire l'étage courant et l'étage cible.
2. en mode actif, quand l'étage cible est atteint, les portes s'ouvrent et le contrôleur passe en mode attente.
3. en mode actif, quand l'etage cible est plus élevé que l'étage courant, le contrôleur fait s'élever l'ascenceur.
4. en mode actif, quand l'étage cible est moins élevé que l'étage courant, le contrôleur fait descendre l'ascenceur.
5. en mode attente, il se peut que quelqu'un entre dans l'ascenceur et choisisse un nouvel étage cible. L'ascenceur ferme alors les portes et redevient actif.
6. initialement, l'ascenceur est à l'étage 0 et en mode attente.

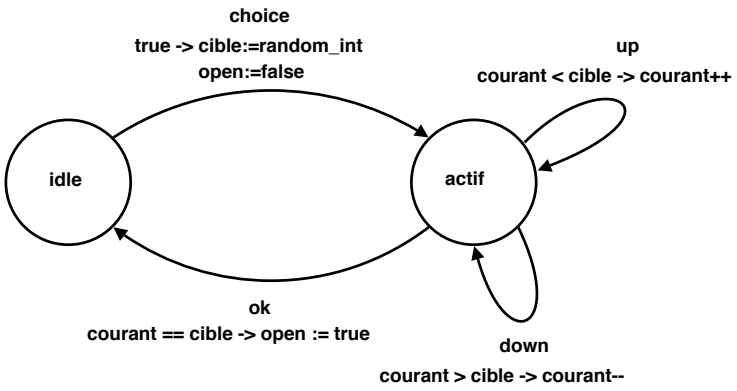
Questions

1. Proposez une machine à états modélisant le contrôleur de l'ascenceur (définition formelle et dessin)
2. Définissez et dessinez le système de transitions correspondant (en vous limitant aux configurations accessibles depuis l'état initial)

The Lift Example

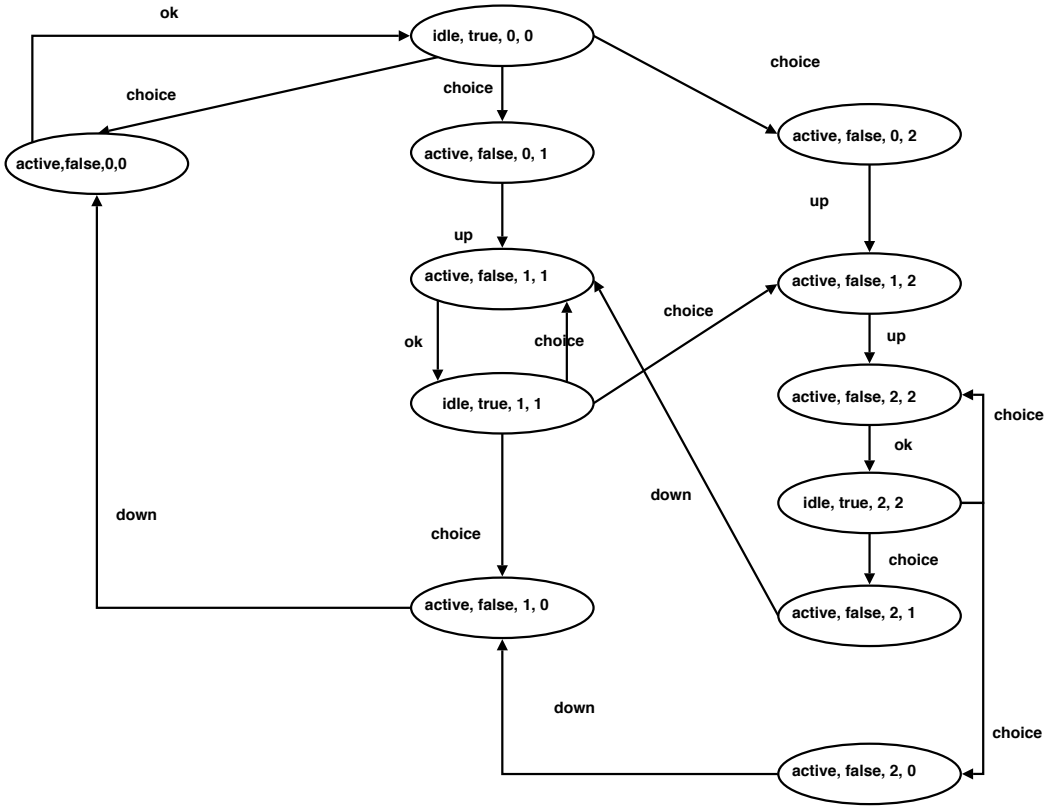
State Machine

- V= `courant:` `int[0...2]`, cible: int[0...2], open: bool
- `random.in` $\in$ `[0...2]`



The Lift Example

Labeled Transition System



Outline

1. Context
2. Model Checking
3. Formalisms and Notations
4. Formal Specifications
   - Petri nets
   - Linear Temporal Logic (LTL)
5. LTL Model Checking
   - Büchi Automata
   - Automata-Theoretic Explicit LTL Model Checking
6. Model Checking CTL
   - Binary Diagram Decision
   - Verification Algorithm for CTL
7. Conclusion

Plan

4 Formal Specifications
- Petri nets
- Linear Temporal Logic (LTL)

Petri Nets [Petri 73]

Syntax

Definition

A Petri net is 5-tuple $N = \langle P, T, F, W, m_0 \rangle$ where:

- P is a finite set of places (cercles) and T a finite set of transitions (squares) with $(P \cup T) \neq \emptyset$ and $P \cap T = \emptyset$,
- A flow relation $F \subseteq (P \times T) \cup (T \times P)$,
- $W : F \to \mathbb{N}^+$ assigns a weight $(> 0)$to any arc.
- An initial marking m_0 where a marking m is a mapping $m : P \to \mathbb{N}$.

Petri Nets [Petri 73]

Syntax

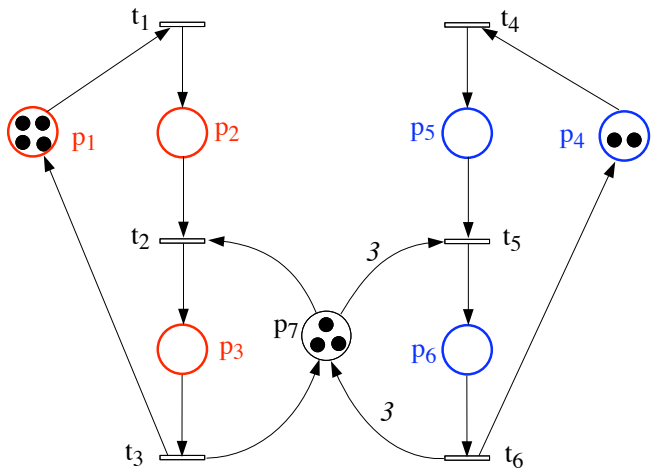
Definition

A Petri net is 5-tuple $N = \langle P, T, F, W, m_0 \rangle$ where:

- P is a finite set of places (cercles) and T a finite set of transitions (squares) with $(P \cup T) \neq \emptyset$ and $P \cap T = \emptyset$,
- A flow relation $F \subseteq (P \times T) \cup (T \times P)$,
- $W : F \to \mathbb{N}^+$ assigns a weight $(> 0)$to any arc.
- An initial marking m_0 where a marking m is a mapping $m : P \to \mathbb{N}$.

The incidence matrix C:
$\forall (p, t) \in P \times T : C(p, t) = W(t, p) - W(p, t)$

Petri Nets: an example



$$\begin{bmatrix} -1 & 0 & 1 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 \\ 0 & -1 & 1 & 0 & -3 & 3 \end{bmatrix}$$

Petri Nets: Semantics

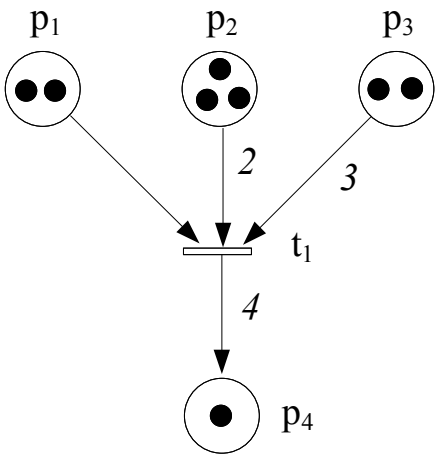
- Fireability of a transition

Petri Nets: Semantics

- Fireability of a transition
  - t is fireable at a marking m iff $\forall p,\ W(p,t) \leq m(p)$

Petri Nets: Semantics

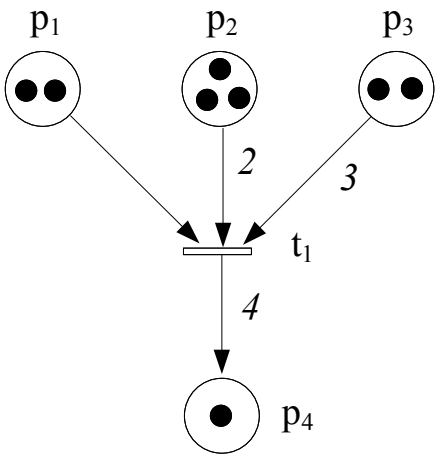
- Fireability of a transition
  - t is fireable at a marking m iff $\forall p,\ W(p,t) \leq m(p)$



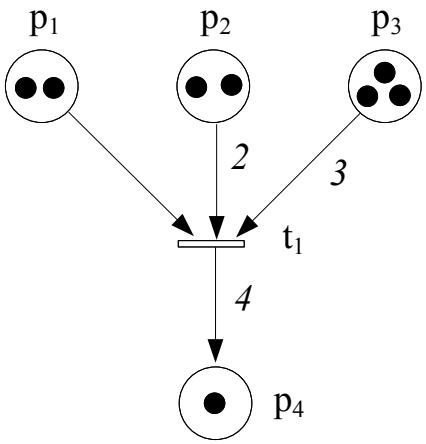
not firable

Petri Nets: Semantics

- Fireability of a transition
  - t is fireable at a marking m iff $\forall p,\ W(p,t) \leq m(p)$



not firable



firable

Petri Nets: Semantics

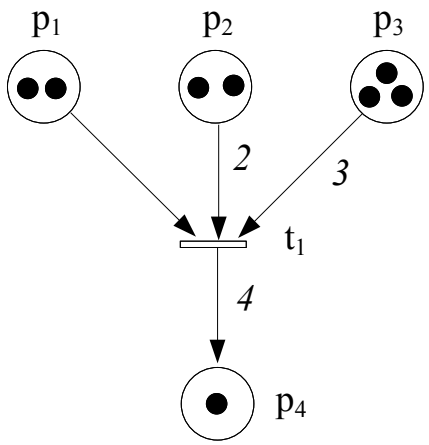
- Firing a transition

Petri Nets: Semantics

- Firing a transition
  - The firing of a (fireable) transition t from a marking m leads to $m' = m - W(p,t) + W(t,p)$

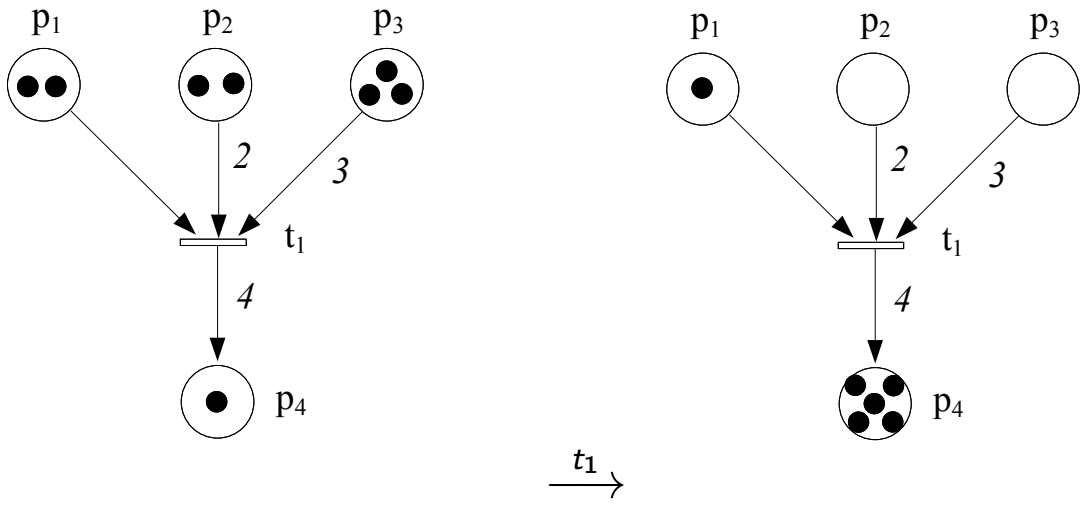
Petri Nets: Semantics

- Firing a transition
  - The firing of a (fireable) transition t from a marking m leads to $m' = m - W(p,t) + W(t,p)$

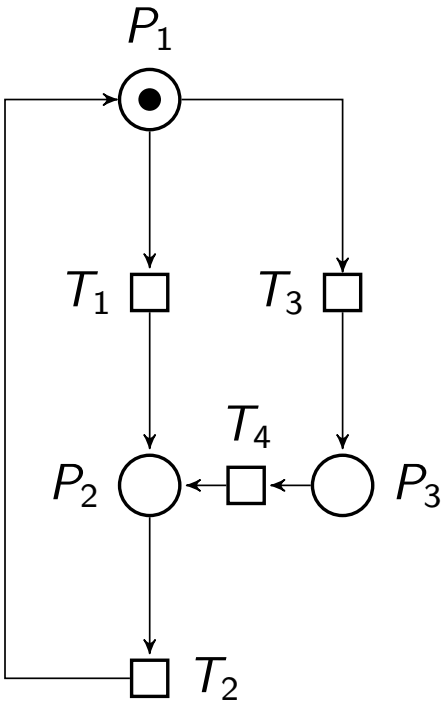


Petri Nets: Semantics

- Firing a transition
  - The firing of a (fireable) transition t from a marking m leads to $m' = m - W(p,t) + W(t,p)$

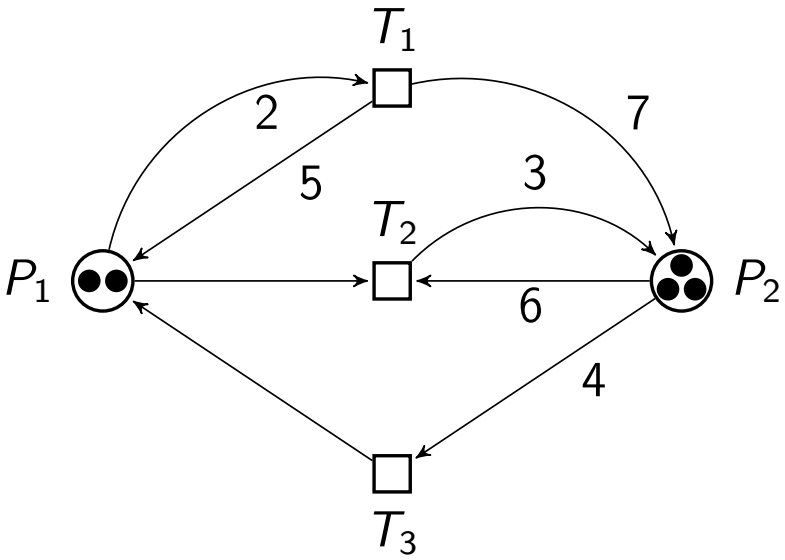


Petri Nets: exercice 1



1. Donner les matrices Pre, Post et celle d'incidence correspondantes.
2. Quelles sont les transitions franchissables à partir du marquage initial?

Petri Nets: exercice 2



1. Est-ce que la transition T_1 est franchissable à partir du marquage initial? Si oui, quel est l'état du réseau après son franchissement?
2. Donner le matrice d'incidence de ce réseau.
3. Vérifier par calcul la franchissabilité de la transition T_1.
4. Calculer la franchissement de T_1 d'abord avec les matrices Pre et Post, ensuite avec la matrice d'incidence.

Petri Nets: Semantics (Cont.)

- $\sigma = t_1 \dots t_n \in T^*$ is fireable at m_0 (denoted by $m_0 \xrightarrow{\sigma}$ iff $\exists m_1 \dots m_n$ s.t. $m_0 \xrightarrow{t_1} m_1 \longrightarrow \dots \xrightarrow{t_n} m_n$

Petri Nets: Semantics (Cont.)

- $\sigma = t_1 \dots t_n \in T^*$ is fireable at m_0 (denoted by $m_0 \xrightarrow{\sigma}$ iff $\exists m_1 \dots m_n$ s.t. $m_0 \xrightarrow{t_1} m_1 \longrightarrow \dots \xrightarrow{t_n} m_n$
- $L(N, m_0) = \{\sigma \in T^* \mid m_0 \xrightarrow{\sigma}\}$

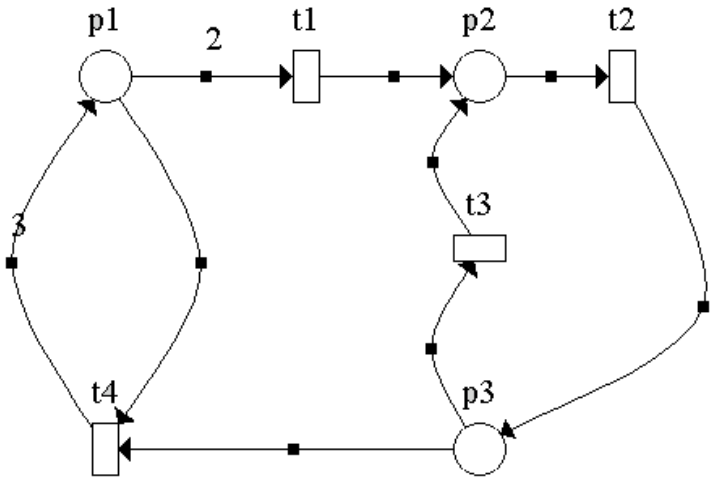
Petri Nets: Semantics (Cont.)

- $\sigma = t_1 \dots t_n \in T^*$ is fireable at m_0 (denoted by $m_0 \xrightarrow{\sigma}$ iff $\exists m_1 \dots m_n$ s.t. $m_0 \xrightarrow{t_1} m_1 \longrightarrow \dots \xrightarrow{t_n} m_n$
- $L(N, m_0) = \{\sigma \in T^* \mid m_0 \xrightarrow{\sigma}\}$
- $R(N, m) =$ the set markings reachable from a marking m of N

Petri Nets: Semantics (Cont.)

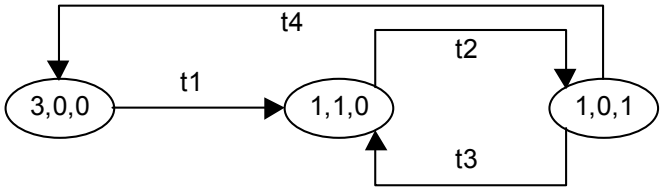
- $\sigma = t_1 \dots t_n \in T^*$ is fireable at m_0 (denoted by $m_0 \xrightarrow{\sigma}$ iff $\exists m_1 \dots m_n$ s.t. $m_0 \xrightarrow{t_1} m_1 \longrightarrow \dots \xrightarrow{t_n} m_n$
- $L(N, m_0) = \{\sigma \in T^* \mid m_0 \xrightarrow{\sigma}\}$
- $R(N, m)$ = the set markings reachable from a marking m of N
- the reachability graph is a LTS $\langle S, A, \rightarrow, s_0 \rangle$ s.t.
  - $S = R(N, m_0)$
  - $A = T$
  - $s_0 = m_0$
  - $(s_1, t, s_2) \in \rightarrow$ iff $s_1 \xrightarrow{t} s_2$

Petri Nets: Reachability Graph

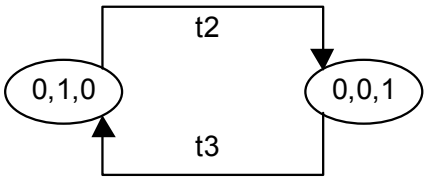


initial marking $(3, 0, 0)$, then $(0, 1, 0)$

Petri Nets: Reachability Graph

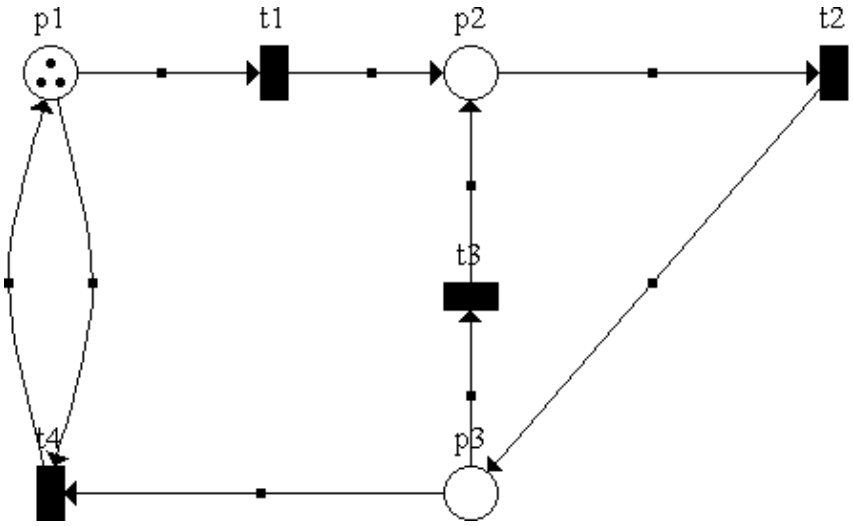


$m_0 = (3, 0, 0)$

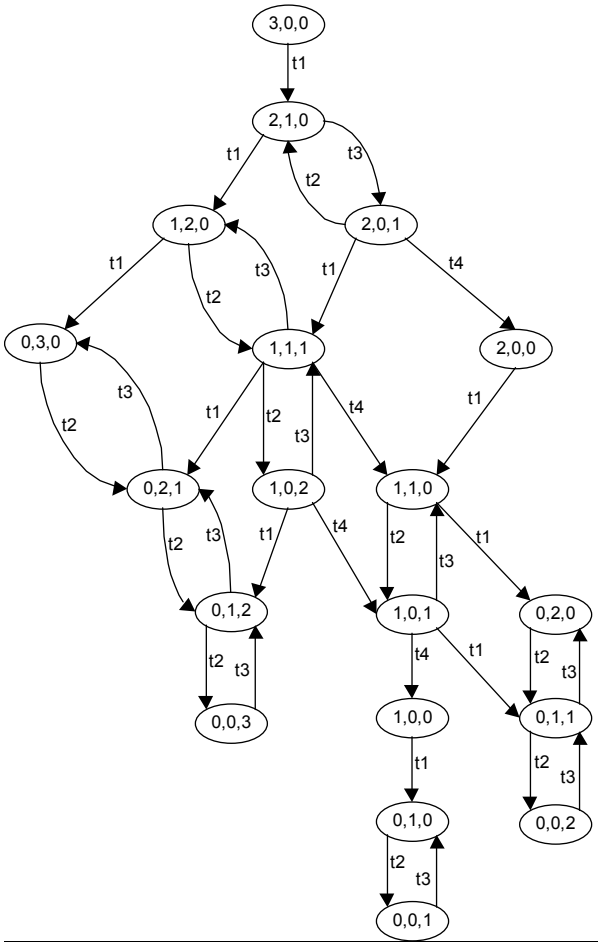


$m_0 = (0, 1, 0)$

Petri Nets: Reachability Graph

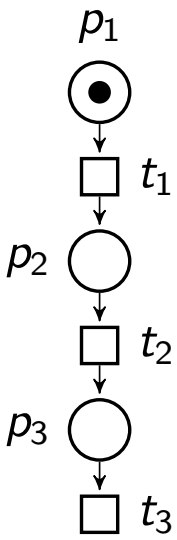


Petri Nets: Reachability Graph



Petri Nets: Expression Power

- Causality

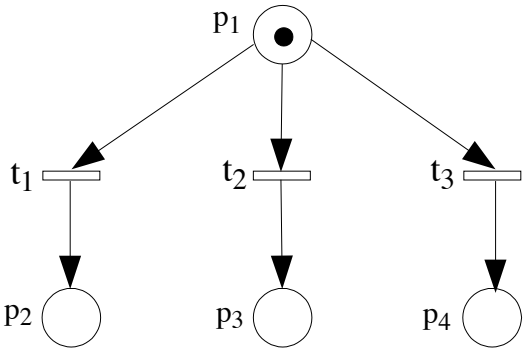


Petri Nets: Expression Power

- Conflict/Choice

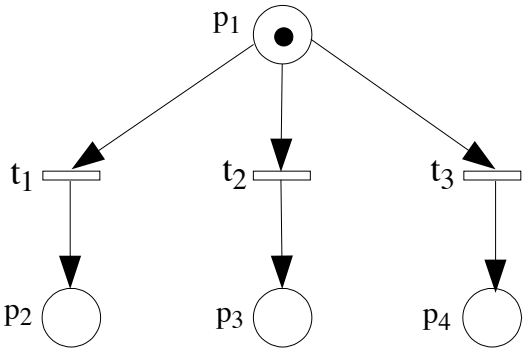
Petri Nets: Expression Power

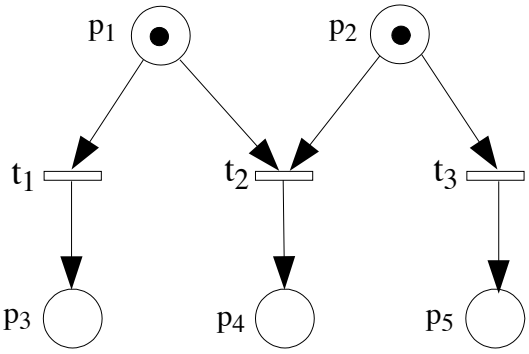
- Conflict/Choice



Petri Nets: Expression Power

- Conflict/Choice



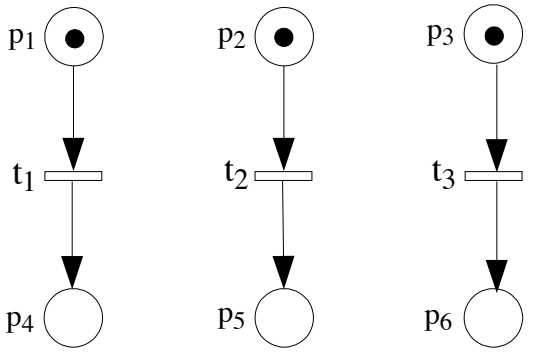


Petri Nets: Expression Power

- Parallelism

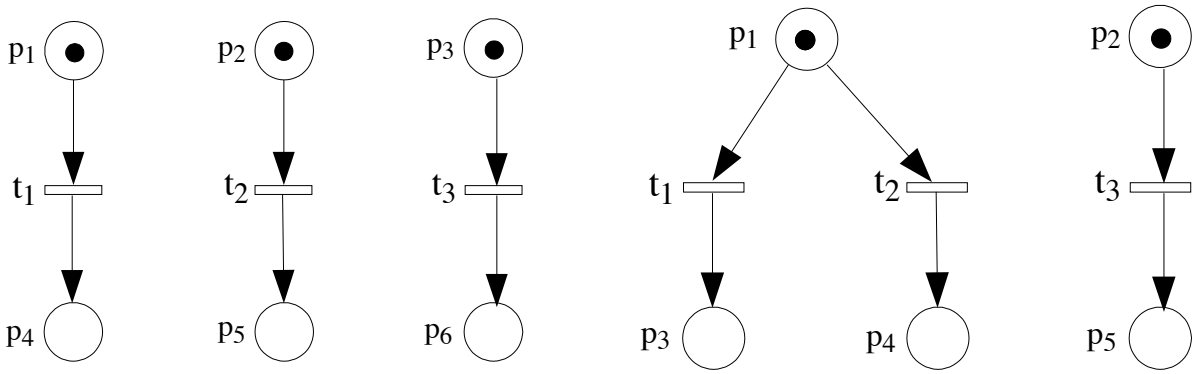
Petri Nets: Expression Power

- Parallelism



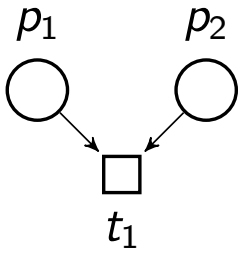
Petri Nets: Expression Power

- Parallelism



Petri Nets: Expression Power

- Synchronization



Example: Mutual Exclusion Algorithm

Global variables: req_P and req_Q

Process P

1. $req_P \leftarrow 1$
2. `wait(`$req_Q = 0$`)`
3. `Critical Section`
4. $req_P \leftarrow 0$

Process Q

1. $req_Q \leftarrow 1$
2. `wait(`$req_P = 0$`)`
3. `Critical Section`
4. $req_Q \leftarrow 0$

Initial state: $req_P = req_Q = 0$

Example: Mutual Exclusion Algorithm

Global variables: req_P and req_Q

Process P

1. $req_P \leftarrow 1$
2. `wait(`$req_Q = 0$`)`
3. `Critical Section`
4. $req_P \leftarrow 0$

Process Q

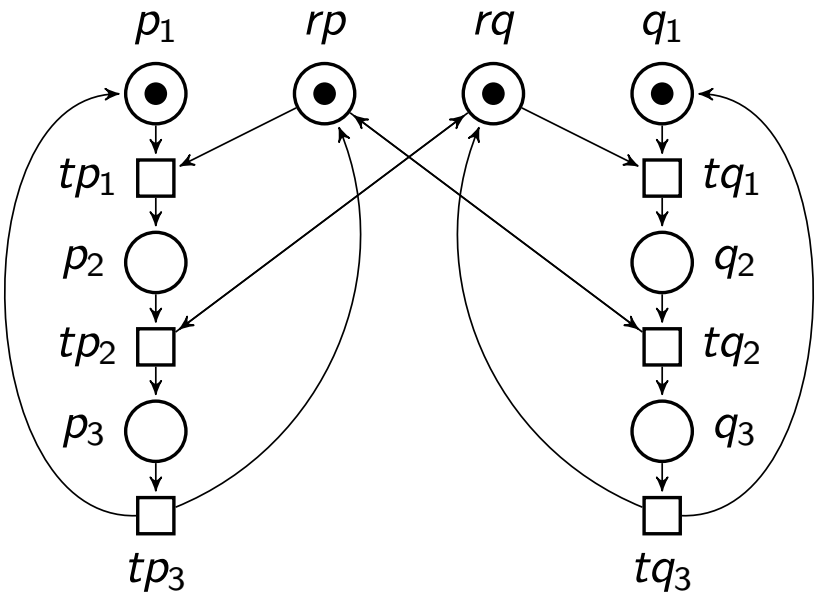
1. $req_Q \leftarrow 1$
2. `wait(`$req_P = 0$`)`
3. `Critical Section`
4. $req_Q \leftarrow 0$

Initial state: $req_P = req_Q = 0$

Properties to be checked:

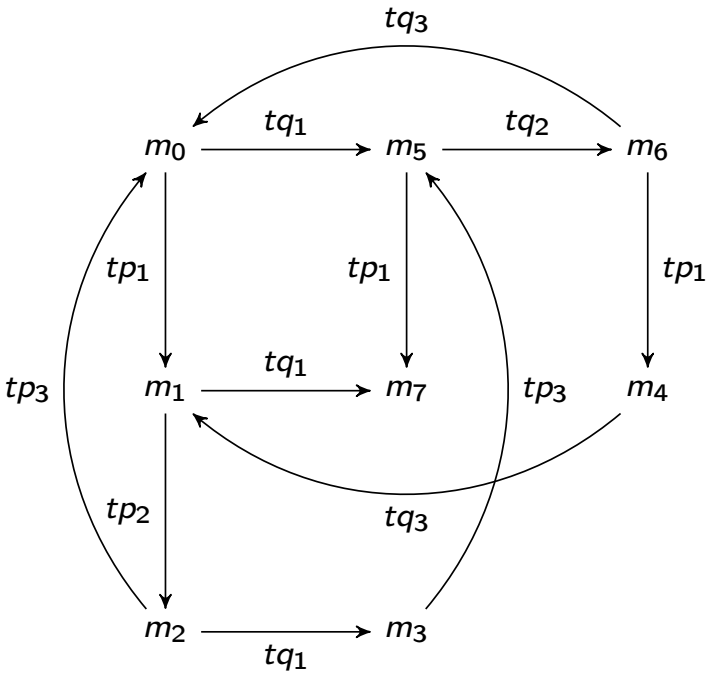
1. Mutual exclusion
2. Fairness
3. Order

Example: Mutual Exclusion Algorithm



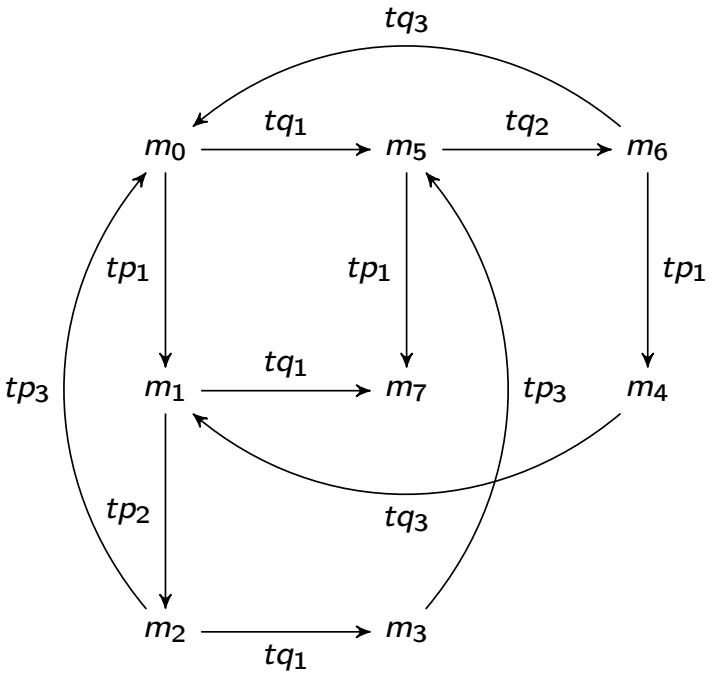
Example: Mutual Exclusion Algorithm

$m_0 = p_1 + rp + rq + q_1$
$m_1 = p_2 + rq + q_1$
$m_2 = p_3 + rq + q_1$
$m_3 = p_3 + q_2$
$m_4 = p_3 + q_2$
$m_5 = p_1 + rp + q_2$
$m_6 = p_1 + rp + q_3$
$m_7 = p_2 + q_2$



Example: Mutual Exclusion Algorithm

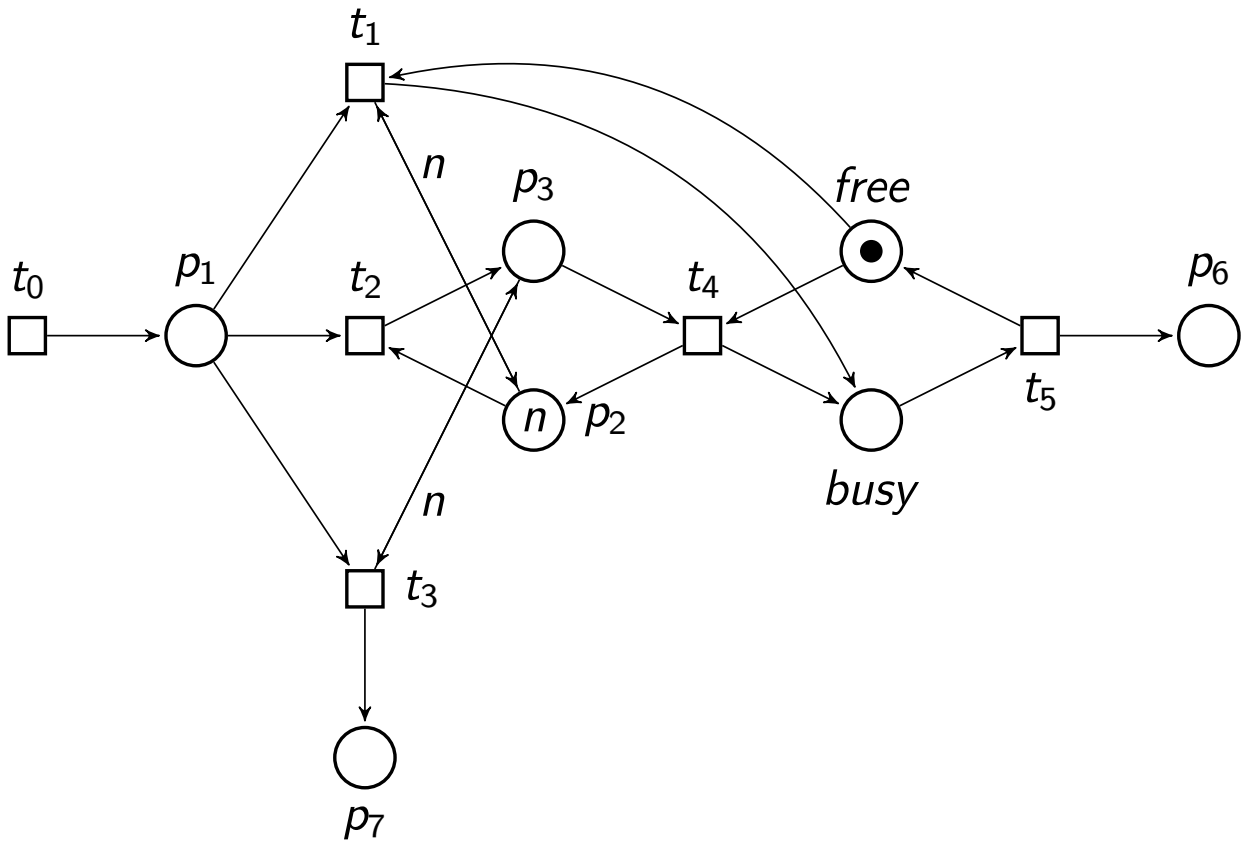
$m_0 = p_1 + rp + rq + q_1$
$m_1 = p_2 + rq + q_1$
$m_2 = p_3 + rq + q_1$
$m_3 = p_3 + q_2$
$m_4 = p_3 + q_2$
$m_5 = p_1 + rp + q_2$
$m_6 = p_1 + rp + q_3$
$m_7 = p_2 + q_2$



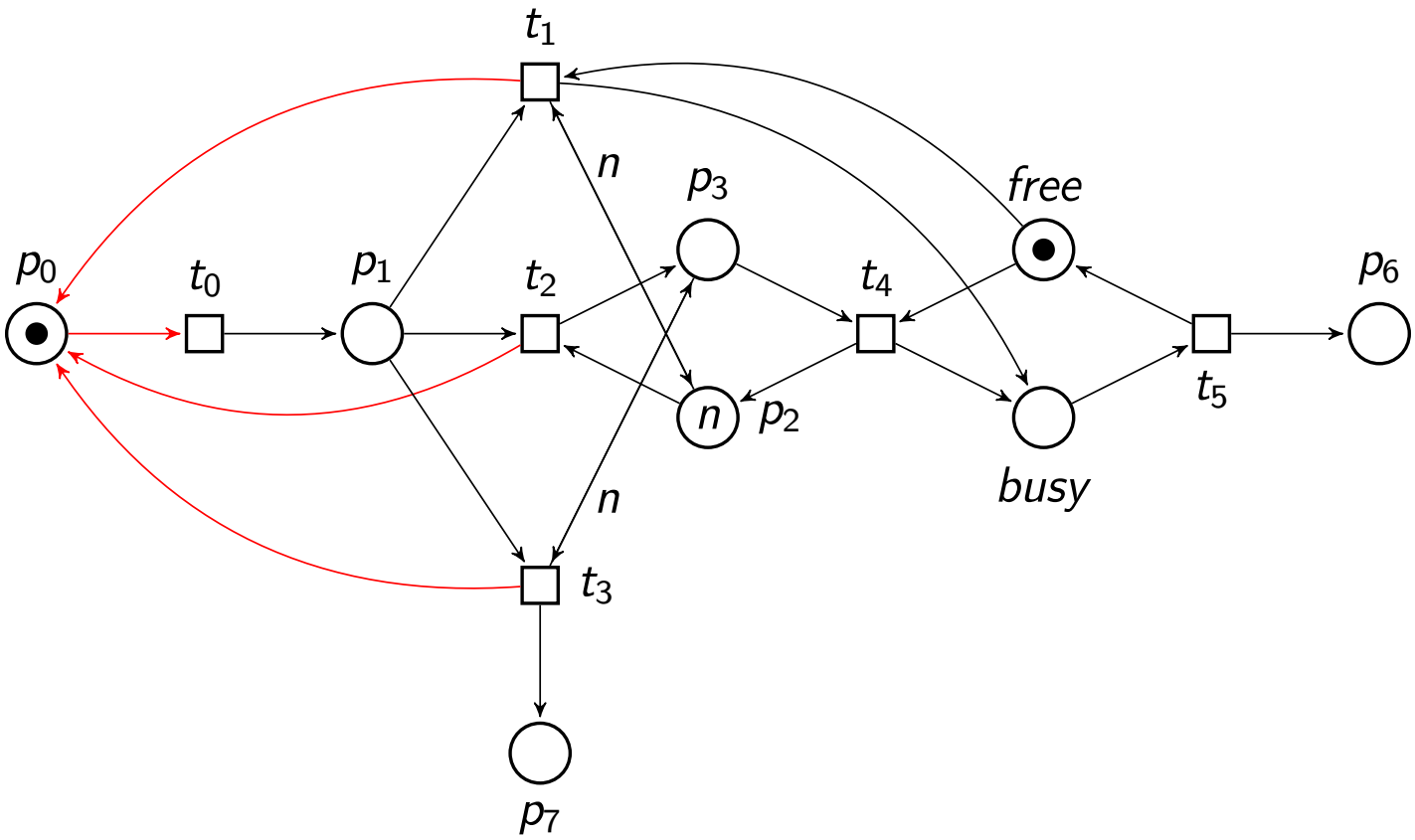
Compare with the previous reachability graph of the mutual exclusion example

Petri Nets modeling a hairdresser

Petri Nets modeling a hairdresser



Petri Nets modeling a hairdresser (Cont.)



Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.
- N is bounded iff $\forall p \in P :\ \exists k \in \mathbb{N}$ s.t. $\forall m \in R(N, m_0),\ m(p) \leq k$.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.
- N is bounded iff $\forall p \in P :\ \exists k \in \mathbb{N}$ s.t. $\forall m \in R(N, m_0),\ m(p) \leq k$.
- N is structurally bounded iff N is bounded for all initial marking m_0.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.
- N is bounded iff $\forall p \in P :\ \exists k \in \mathbb{N}$ s.t. $\forall m \in R(N, m_0),\ m(p) \leq k$.
- N is structurally bounded iff N is bounded for all initial marking m_0.
- N is quasi-live iff $\forall t \in T :\ \exists M \in R(N, m_0)$ for which t is enabled.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.
- N is bounded iff $\forall p \in P :\ \exists k \in \mathbb{N}$ s.t. $\forall m \in R(N, m_0),\ m(p) \leq k$.
- N is structurally bounded iff N is bounded for all initial marking m_0.
- N is quasi-live iff $\forall t \in T :\ \exists M \in R(N, m_0)$ for which t is enabled.
- N is deadlock-free (weakly live) iff $\forall M \in R(N, m_0),\ \exists t \in T$ enabled in M.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.
- N is bounded iff $\forall p \in P :\ \exists k \in \mathbb{N}$ s.t. $\forall m \in R(N, m_0),\ m(p) \leq k$.
- N is structurally bounded iff N is bounded for all initial marking m_0.
- N is quasi-live iff $\forall t \in T :\ \exists M \in R(N, m_0)$ for which t is enabled.
- N is deadlock-free (weakly live) iff $\forall M \in R(N, m_0),\ \exists t \in T$ enabled in M.
- N is live iff $\forall t \in T :\ \forall m \in R(N, m_0) \exists m' \in R(N, m)$ for which t is enabled.

Petri Nets: Properties

- A marking m^* is a home state if and only if $\forall m \in R(N, m_0),\ m^* \in R(N, m)$.
- N is reversible iff m_0 is a home state.
- N is bounded iff $\forall p \in P :\ \exists k \in \mathbb{N}$ s.t. $\forall m \in R(N, m_0),\ m(p) \leq k$.
- N is structurally bounded iff N is bounded for all initial marking m_0.
- N is quasi-live iff $\forall t \in T :\ \exists M \in R(N, m_0)$ for which t is enabled.
- N is deadlock-free (weakly live) iff $\forall M \in R(N, m_0),\ \exists t \in T$ enabled in M.
- N is live iff $\forall t \in T :\ \forall m \in R(N, m_0) \exists m' \in R(N, m)$ for which t is enabled.
- N is structurally live iff $\forall m_0, (N, m_0)$ is live.

Relation Between Properties

- quasi-liveness VS Liveness ??

Relation Between Properties

- quasi-liveness VS Liveness ??
- quasi-liveness VS weak liveness ??

Relation Between Properties

- quasi-liveness VS Liveness ??
- quasi-liveness VS weak liveness ??
- liveness VS weak liveness ??

Relation Between Properties

- quasi-liveness VS Liveness ??
- quasi-liveness VS weak liveness ??
- liveness VS weak liveness ??
- m_0 home state and quasi live $\Rightarrow$ live ?? (if yes, proof)